

Relational Databases (SQL)

Java Backend Interview — Short Notes

1. RDBMS Fundamentals

- Relational model: data stored in **tables (relations)**. Rows = tuples, Columns = attributes.
- **Schema**: structure definition — tables, columns, data types, constraints, relationships.
- **Primary Key (PK)**: uniquely identifies each row. Cannot be NULL. Usually auto-increment or UUID.
- **Foreign Key (FK)**: references PK of another table. Enforces referential integrity.
- **Unique constraint**: column value must be unique across all rows (NULLs allowed per SQL standard).
- **Check constraint**: custom row-level validation (e.g., age > 0).
- **Index**: data structure (usually B-Tree) speeding up reads. Slows writes. Created automatically on PK/UK.

1.1 ACID Properties

Property	Description	Realistic Example
Atomicity	Transaction is all-or-nothing. If any step fails, entire transaction rolls back.	Bank transfer: debit account A + credit account B. If credit fails, debit is also rolled back.
Consistency	DB moves from one valid state to another. All constraints, rules, and triggers must hold.	Cannot insert order with non-existent customer_id (FK constraint prevents it).
Isolation	Concurrent transactions behave as if sequential. Intermediate states not visible to others.	Two users booking last flight seat simultaneously — isolation ensures only one succeeds.
Durability	Committed transaction survives crashes. Changes written to durable storage (WAL/redo log).	After payment confirmed, power outage does not lose the transaction record.

2. Normalization

Normal Form	Rule	Problem It Solves	Example
1NF	Atomic values, no repeating groups, each row unique.	Eliminates multi-valued cells.	phone='123,456' → split into separate rows.
2NF	In 1NF + no partial dependency (non-key column depends on whole PK).	Eliminates data duplication in composite-key tables.	OrderItem(order_id, product_id, product_name) → product_name depends only on product_id, move to Product table.
3NF	In 2NF + no transitive dependency (non-key column depends on another non-key column).	Eliminates derived data stored redundantly.	Employee(id, dept_id, dept_name) → dept_name depends on dept_id, move to Department table.
BCNF	Stricter 3NF — every determinant must be a candidate key.	Handles edge cases with overlapping candidate keys.	Booking(student, subject, teacher) where teacher→subject — decompose further.

✓ Denormalization: intentionally break normal forms for read performance. Add redundant columns or derived data. Trade space/write complexity for query speed.

3. Transactions & Locking

3.1 Transaction Isolation Levels

Level	Dirty Read	Non-Repeatable Read	Phantom Read	Description & Use Case
READ UNCOMMITTED	Yes	Yes	Yes	Can read uncommitted changes from other txns. Almost never used. Use: metrics aggregation where slight inaccuracy is fine.
READ COMMITTED	No	Yes	Yes	Default in PostgreSQL, Oracle, SQL Server. Only see committed data. Re-reading same row may return different values if another txn committed in between. Use: most OLTP workloads.
REPEATABLE READ	No	No	Yes	Default in MySQL InnoDB. Same row reads always return same value within txn. Phantom rows (new rows matching WHERE) still possible. Use: financial reports, consistent reads.
SERIALIZABLE	No	No	No	Full isolation — transactions execute as if sequential. Range locks prevent phantoms. Slowest. Use: seat booking, inventory reservation, banking.

3.2 Locking

Lock Type	Description	Compatibility	Realistic Use Case
Shared (S) Lock	Read lock. Multiple txns can hold simultaneously.	Compatible with other S locks. Blocked by X lock.	SELECT ... FOR SHARE — reading a row that others can also read but not modify.
Exclusive (X) Lock	Write lock. Only one txn can hold at a time.	Incompatible with S and X locks.	UPDATE or DELETE — prevents any other read or write during modification.
Intent Locks (IS/IX)	Table-level signal that row-level locks exist. Prevents table lock conflicts.	IX blocks table S lock. IS blocks table X lock.	InnoDB automatically sets intent locks before row locks.
Row-level Lock	Lock only the specific row. High concurrency.	Other rows unaffected.	UPDATE orders SET status='shipped' WHERE id=5 — only locks row 5.
Table-level Lock	Lock entire table. Low concurrency, fast acquire.	Blocks all other access.	LOCK TABLE orders WRITE — during schema migration or bulk load.
Gap Lock	Lock gap between index values. Prevents phantom rows.	Held with row lock in REPEATABLE READ.	MySQL InnoDB prevents INSERT of rows matching your WHERE range.
Deadlock	Two txns each waiting for lock held by the other.	DB detects and kills one txn (victim).	Txn A locks row 1 then row 2. Txn B locks row 2 then row 1 — circular wait.

• **Optimistic Locking:** no DB lock — use version column. On update check version hasn't changed. Retry on conflict. Good for low-contention.

• **Pessimistic Locking:** SELECT FOR UPDATE acquires X lock immediately. Good for high-contention.

```
-- Pessimistic: lock row before updating
BEGIN;
SELECT * FROM accounts WHERE id=1 FOR UPDATE;
UPDATE accounts SET balance=balance-100 WHERE id=1;
COMMIT;
```

```
-- Optimistic: check version on update
UPDATE orders SET status='shipped', version=version+1
WHERE id=5 AND version=3; -- 0 rows = someone else updated first
```

4. Indexing

4.1 Index Types

Index Type	Structure	Best For	Notes
B-Tree	Balanced tree — sorted. Default for most DBs.	Equality (=), range (<,>,BETWEEN), ORDER BY, prefix LIKE.	Most versatile. Used for PK, UK, FK by default.
Hash	Hash table — direct key lookup.	Exact equality only (=). Very fast O(1).	Cannot handle ranges or ordering. PostgreSQL supports. InnoDB does not.
Composite (Multi-col)	B-Tree on multiple columns.	Queries on leading columns of index.	Column order matters. (a,b,c) index helps WHERE a=? and WHERE a=? AND b=? but not WHERE b=? alone.
Covering Index	Index includes all columns needed by query.	Queries that can be satisfied from index alone — no table row access.	SELECT email FROM users WHERE status='active' — index on (status, email).
Partial Index	Index on subset of rows (WHERE condition).	Frequently queried subset. Smaller, faster.	CREATE INDEX ON orders(id) WHERE status='pending' — only indexes pending orders.
Full-Text	Inverted index of words/tokens.	Text search (MATCH/CONTAINS).	Postgres tsvector/tsquery, MySQL FULLTEXT. Not for LIKE '%term%'.

■ **Over-indexing:** every index slows INSERT/UPDATE/DELETE. Index only columns used in WHERE, JOIN ON, ORDER BY, GROUP BY.

■ **Low cardinality columns** (boolean, status with 3 values) benefit little from indexes — DB may ignore them.

4.2 Query Execution & EXPLAIN

- **EXPLAIN / EXPLAIN ANALYZE:** shows query plan — seq scan vs index scan, join type, row estimates, actual cost.
- **Seq Scan:** full table scan. Fine for small tables or fetching >15% rows. Bad on large tables.
- **Index Scan:** traverse B-Tree then fetch table row. Good for low selectivity (few rows returned).
- **Index-Only Scan:** all data in index (covering). No table access — fastest.
- **Nested Loop Join:** for each row in outer table, scan inner. Good for small outer + indexed inner.
- **Hash Join:** build hash table from smaller table, probe with larger. Good for large equi-joins.
- **Merge Join:** both sides sorted — merge. Good for sorted or pre-sorted inputs.

5. Essential SQL

5.1 JOINS

JOIN Type	Returns	Use Case
INNER JOIN	Rows where join condition matches in BOTH tables.	Orders with their customers — exclude orders with no matching customer.
LEFT OUTER JOIN	All rows from left table + matching right rows (NULL if no match).	All customers, including those with no orders. Right side NULLs = no match.

RIGHT OUTER JOIN	All rows from right table + matching left rows (NULL if no match).	Rarely used — rewrite as LEFT JOIN with tables swapped.
FULL OUTER JOIN	All rows from both tables. NULLs on either side where no match.	Find unmatched rows on both sides — data reconciliation.
CROSS JOIN	Cartesian product — every combination. N*M rows.	Generate all size/color combinations for a product matrix.
SELF JOIN	Table joined to itself.	Org hierarchy — employee to manager: JOIN employees mgr ON e.manager_id=mgr.id.

5.2 Window Functions

- Compute aggregates *across a set of rows related to current row* without collapsing rows.

```
SELECT employee_id, salary,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_rank,
       LAG(salary) OVER (PARTITION BY dept_id ORDER BY hire_date) AS prev_salary,
       SUM(salary) OVER (PARTITION BY dept_id) AS dept_total,
       AVG(salary) OVER (ORDER BY hire_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS rolling_avg
FROM employees;
```

Function	Purpose	Example
ROW_NUMBER()	Unique sequential number per partition.	Paginate results per department.
RANK()	Rank with gaps for ties (1,1,3).	Leaderboard with tied positions skipped.
DENSE_RANK()	Rank without gaps for ties (1,1,2).	Top N per group without skipping ranks.
LAG/LEAD	Access previous/next row's value.	Month-over-month revenue change.
FIRST_VALUE/LAST_VALUE	First or last value in window frame.	Running max salary in department.
NTILE(n)	Divide rows into n equal buckets.	Quartile analysis, percentile grouping.

5.3 CTEs & Subqueries

```
-- CTE: readable, reusable within query
WITH monthly_revenue AS (
  SELECT DATE_TRUNC('month', order_date) AS month,
         SUM(total) AS revenue
  FROM orders GROUP BY 1
),
ranked AS (
  SELECT *, RANK() OVER (ORDER BY revenue DESC) AS rk FROM monthly_revenue
)
SELECT * FROM ranked WHERE rk <= 3;

-- Recursive CTE: org hierarchy traversal
WITH RECURSIVE hierarchy AS (
  SELECT id, name, manager_id, 0 AS level FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, h.level+1
  FROM employees e JOIN hierarchy h ON e.manager_id=h.id
)
SELECT * FROM hierarchy ORDER BY level;
```

6. Performance & Realistic Use Cases

6.1 Common Performance Patterns

- **Pagination:** OFFSET/LIMIT is slow for large offsets (scans N rows). Use keyset/cursor pagination:

-- Keyset pagination — fast at any page

```
SELECT * FROM orders WHERE (created_at, id) < (:last_ts, :last_id)
ORDER BY created_at DESC, id DESC LIMIT 20;
```

- **N+1 prevention:** use JOIN instead of loop queries. Use batch IN clause for IDs.
- **Connection pooling:** HikariCP (Spring Boot default). Never open raw connections per request.
- **Read replicas:** route SELECT to replica, INSERT/UPDATE/DELETE to primary.
- **Partitioning:** split large tables by range (date) or hash. Each partition is a separate file.

6.2 Realistic Use Case Scenarios

Use Case	Tables/Schema	Key Technique
E-commerce order	users, products, orders, order_items, inventory	Transaction for stock deduction + order creation. Optimistic lock on inventory.
Banking transfer	accounts(id, balance, version)	SERIALIZABLE isolation or SELECT FOR UPDATE. Debit+credit in same atomic transaction.
Leaderboard	scores(user_id, score, game_id, created_at)	Composite index (game_id, score DESC). Window RANK(). Cache top-N in Redis.
Audit trail	audit_log(table_name, row_id, action, old_val, new_val, changed_by, ts)	Triggers or application-level logging. Partitioned by month for performance.
Report query	Large fact + dimension tables	Aggregation with GROUP BY. Covering indexes. Materialized views for pre-computed results.
Soft delete	deleted_at TIMESTAMP NULL column	Partial index WHERE deleted_at IS NULL. Filter in every query or use view.

7. Interview Questions

Fundamentals & ACID

Q1. What is ACID? Why does it matter?

→ ACID guarantees correctness of transactions. Atomicity: all-or-nothing. Consistency: constraints always maintained. Isolation: concurrent transactions don't interfere. Durability: committed data survives crashes. Without ACID: double charges, lost updates, corrupted data in bank transfers.

Q2. What are isolation levels? Which is default in PostgreSQL vs MySQL?

→ 4 levels: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE. PostgreSQL default: READ COMMITTED. MySQL InnoDB default: REPEATABLE READ. Higher isolation = more locks, less concurrency, fewer anomalies.

Q3. Explain dirty read, non-repeatable read, phantom read.

→ Dirty read: reading uncommitted data from another transaction. Non-repeatable read: re-reading same row gives different result (another txn committed). Phantom read: re-executing a range query returns new rows (another txn inserted). Each resolved by progressively stronger isolation.

Q4. What is a deadlock? How to prevent it?

→ Circular lock dependency — Txn A holds lock X waits for Y, Txn B holds Y waits for X. DB detects and rolls back one. Prevention: always acquire locks in same order, keep transactions short, use row-level locks, retry on deadlock.

Indexing & Performance

Q5. When should you NOT create an index?

→ Low-cardinality column (boolean — only 2 values), small tables (full scan faster), columns rarely used in WHERE/JOIN/ORDER BY, write-heavy tables (every write updates all indexes), columns with frequent updates.

Q6. What is a covering index?

→ Index that contains all columns needed by a query — query satisfied from index alone without accessing the table rows. Fastest possible read. Example: `SELECT email FROM users WHERE status='active'` — index on (status, email) covers both filtering and projection.

Q7. How does a composite index work? Column order matters?

→ B-Tree on multiple columns in specified order. Index (a,b,c) helps: `WHERE a=?`, `WHERE a=? AND b=?`, `WHERE a=? AND b=? AND c=?` but NOT `WHERE b=?` alone or `WHERE c=?`. Leading column must be in WHERE for index to be used.

Q8. Explain EXPLAIN ANALYZE output.

→ Shows query plan: Seq Scan (full scan), Index Scan (B-tree traversal + row fetch), Index Only Scan (from index alone), Nested Loop/Hash/Merge Join. Key metrics: rows estimated vs actual, cost, loops. High 'rows estimated vs actual' mismatch = stale statistics — run ANALYZE.

Design & Normalization

Q9. When to denormalize?

→ When read performance is critical and joins are too slow. Example: reporting/analytics tables, frequently queried aggregates. Trade-off: redundant data, more complex writes, stale data risk. Always start normalized, denormalize only when profiling shows bottleneck.

Q10. Difference between clustered and non-clustered index?

→ Clustered: physical order of table rows matches index order. Only one per table. InnoDB: PK is always clustered. Reading clustered index = reading the rows directly. Non-clustered: separate structure pointing to row locations. Multiple allowed. Extra lookup to fetch row data.

Q11. How to design for soft deletes?

→ Add `deleted_at` TIMESTAMP NULL column. Filter with `WHERE deleted_at IS NULL` in every query (or create a view). Create partial index `WHERE deleted_at IS NULL` for performance. Consider archiving to separate table after N months to keep main table clean.

Q12. Optimistic vs pessimistic locking?

→ Optimistic: no DB lock — version column checked on UPDATE. If mismatch, retry. Good for low-contention reads. Pessimistic: `SELECT FOR UPDATE` acquires row lock immediately. Other transactions wait. Good for high-contention critical sections (inventory reservation, seat booking).

Advanced

Q13. What is a materialized view?

→ Pre-computed query result stored as a table. Reads are instant (no query computation). Must be refreshed when base data changes (`REFRESH MATERIALIZED VIEW`). Use for: expensive aggregation reports, dashboards, read-heavy derived data.

Q14. What is connection pooling and why is it needed?

→ Database connections are expensive to open (TCP handshake, auth, session setup). Pool maintains pre-opened connections and reuses them. HikariCP: `minimum-idle`, `maximum-pool-size`. Without pooling: each request opens/closes connection — massive overhead at scale.

Q15. How does a B-Tree index work?

→ Balanced tree: root → internal nodes → leaf nodes. Each internal node has sorted keys + child pointers. Leaf nodes have keys + row pointers (or row data for clustered). Search: $O(\log N)$. Range scan: traverse leaf linked list. Inserted key may cause node split (rebalance).